

FleetManager Project Plan

1. Introduzione

1.1 Background e storia del progetto

FleetManager nasce come progetto accademico per il corso di Ingegneria del Software A.A. 2025/26 (Prof. Angelo Gargantini, Dott.ssa Silvia Bonfanti), con l'obiettivo di applicare in modo pratico le metodologie di analisi, progettazione e gestione studiate a lezione.

Il progetto si colloca all'interno del percorso di valutazione del corso e costituisce parte integrante dell'esame finale. Tutta la documentazione, i diagrammi e il codice sorgente vengono mantenuti in un repository GitHub condiviso con i docenti (garganti, silviabonfanti) e aggiornato progressivamente nel corso dello sviluppo.

FleetManager nasce dall'esigenza di disporre di un sistema software per la gestione centralizzata di flotte aziendali, che consenta di amministrare i veicoli, gestire le prenotazioni dei dipendenti e monitorare scadenze e manutenzioni.

Il progetto adotta le tecnologie e i vincoli richiesti dal corso:

- Java 17 come linguaggio di programmazione;
- Eclipse come IDE;
- Maven per la gestione delle dipendenze;
- Papyrus UML per la modellazione;
- JUnit 5 per i test;
- Database H2 embedded;
- JavaFX 21 per la parte grafica;
- GitHub per la collaborazione e il versionamento.

Come previsto dalle linee guida, il Project Plan viene consegnato circa un mese prima dell'esame, mentre il progetto completo deve essere finalizzato cinque giorni prima della presentazione.

1.2 Obiettivi del progetto

- Realizzare un sistema gestionale completo per flotte aziendali che permetta:
 - la gestione dell'anagrafica dei veicoli,
 - la prenotazione e restituzione da parte dei dipendenti,
 - la pianificazione delle manutenzioni e il monitoraggio delle scadenze (bollo, assicurazione, revisione).
- Applicare un processo di sviluppo Agile iterativo con sprint brevi e consegne incrementali.

- Utilizzare in modo integrato UML → Java → JUnit, garantendo tracciabilità tra requisiti, design e test.
- Dimostrare la padronanza degli strumenti professionali previsti (Eclipse, Maven, GitHub, Papyrus).
- Produrre documentazione completa e conforme al modello Van Vliet (14 punti).

1.3 Risultati attesi

- Repository GitHub con la struttura richiesta (documenti/ + codice/) e uso effettivo di issue, branch e pull request.
- Progetto Eclipse/Maven funzionante, privo di riferimenti assoluti, con test automatizzati JUnit.
- Progetto Papyrus UML contenente i diagrammi richiesti (use case, class, state, sequence, activity, component, package).
- Documentazione completa in PDF: Project Plan, Gestione progetto, Requisiti, Design, Testing, Maintenance.
- Presentazione PowerPoint per l'esame orale e demo eseguibile del sistema.

1.4 Nomi delle persone responsabili

Il team di sviluppo è composto da due membri, che collaborano in tutte le fasi progettuali, condividendo responsabilità e versionamento:

- Francesco Basis: 1092076 – f.basis@studenti.unibg.it
- Pietro Plati: 1091949 – p.plati@studenti.unibg.it

1.5 Sintesi del progetto

FleetManager è una soluzione software progettata per semplificare la gestione operativa di una flotta aziendale, gestendo prenotazioni, controlli di stato e promemoria di manutenzione. Il progetto combina modellazione UML, programmazione Java, testing automatizzato e versionamento collaborativo GitHub per produrre un sistema completo, funzionante e documentato.

2. Modello di processo

2.1 Scelta del modello

Per FleetManager è stato scelto un modello di processo Agile iterativo–incrementale, ispirato al framework Scrum ma adattato alle dimensioni ridotte del team (2 membri: Pietro Plati e Francesco Basis).

L'obiettivo è garantire flessibilità, feedback continuo e integrazione progressiva dei moduli, riducendo i rischi e mantenendo una documentazione sempre aggiornata.

Il processo Agile è preferito rispetto al modello a cascata perché:

- i requisiti possono evolversi man mano che emergono nuove necessità o vincoli tecnici;
- la frequenza delle revisioni intermedie consente di verificare il raggiungimento delle pietre miliari e la qualità del prodotto;
- permette di testare funzionalità complete a ogni incremento, riducendo il rischio di integrazione finale.

2.2 Struttura del processo

Il ciclo di sviluppo prevede sprint brevi (circa 1 settimana cadauno), ciascuno con obiettivi chiari, deliverable verificabili e test JUnit associati.

Ogni sprint include le fasi fondamentali del ciclo di vita software:

1. Pianificazione (definizione obiettivi e requisiti incrementali)
2. Progettazione UML (aggiornamento dei diagrammi in Papyrus)
3. Implementazione e integrazione del codice Java/Maven
4. Verifica e test (unit test JUnit e review GitHub)
5. Retrospettiva e aggiornamento documentazione

2.3 Pietre Miliari

Milestone	Descrizione	Deliverable
M0 – Setup iniziale	Creazione repository GitHub, configurazione progetto Maven, setup database H2 e baseline Papyrus	Repository iniziale, struttura Maven, pom.xml, .project, configurazione H2 funzionante
M1 – Modellazione base	Sviluppo Use Case Diagram e Class Diagram principali, definizione requisiti IEEE 830	File .uml Papyrus, documento requisiti, modello concettuale approvato.
M2 – Funzionalità core	Implementazione CRUD per Utente e Veicolo, definizione DAO e Service layer, primi test JUnit	Codice Java funzionante (DAO + Service), test unitari, persistenza H2 valida.
M3 – Prenotazioni e stati veicolo	Logica prenotazioni (richiesta/conferma), aggiornamento stato veicolo, integrazione State Machine Diagram, prime schermate JavaFX 21 con SceneManager	Service prenotazioni completo, State Machine aggiornato, UI base JavaFX 21.
M4 – Scadenze, manutenzioni e notifiche automatiche	Gestione scadenze (bollo, revisione, assicurazione), implementazione manutenzioni, generazione notifiche automatiche, aggiornamento UI	Codice Java completo (scadenze + manutenzioni + notifiche), test JUnit, logging Log4j2, UI aggiornata.
M5 – Refactoring, metriche e stabilizzazione	Refactoring architetturale, miglioramento qualità codice, misure SonarLint/PMD/ Stan4J, aggiornamento UML, aumento copertura test	Report metriche, codice refattorizzato, UML aggiornato, documentazione finale.
M6 – Consegna finale e demo	Finalizzazione documentazione (project plan, requisiti, design, testing, maintenance), preparazione presentazione e demo	Repository finale completo, documentazione PDF, slide presentazione, demo JavaFX 21 funzionante.

2.4 Criteri di verifica delle pietre miliari

Ogni milestone è considerata raggiunta se:

- tutti i test JUnit associati superano la verifica;
- i diagrammi UML sono aggiornati nel repository;

- la documentazione è coerente con l'implementazione.

2.5 Percorso critico

Il percorso critico del progetto coincide con la modellazione iniziale e la prima integrazione funzionale (M1–M3). Eventuali ritardi in queste fasi comportano slittamenti su tutte le successive attività (testing, metriche, refactoring).

Per questo motivo:

- i diagrammi UML saranno sviluppati in modo iterativo già dallo Sprint 0;
- i test verranno creati in parallelo all'implementazione;
- la documentazione sarà aggiornata incrementando la stessa base GitHub.

2.6 Pianificazione temporale indicativa

Sprint	Durata	Attività principali	Deliverable
Sprint 0	1 settimana	Setup GitHub, Maven, Papyrus, creazione progetto e documenti iniziali (README, Project Plan v1.0, Changelog)	Repo + progetto base
Sprint 1	1 settimana	Modellazione Use Case Diagram, attori, casi d'uso Driver/Manager, prime analisi dei requisiti	Use Case Diagram + Requisiti v1.0
Sprint 2	1 settimana	Creazione Class Diagram, integrazioni iterative, definizione package model e getter/setter	Class Diagram completo + modello aggiornato
Sprint 3	1 settimana	Implementazione package repository, creazione DAO, prime classi model, configurazione DB H2 e tabelle	DAO iniziali + H2 funzionante
Sprint 4	1 settimana	Popolamento DB (Seeder + JSON), aggiunta Foreign Key, creazione DatabaseManager, primi test JUnit su DAO	DB completo + primi test JUnit
Sprint 5	1 settimana	Creazione Service layer: GestoreLogin, GestorePrenotazioni, GestoreScadenze, GestoreManutenzioni + test relativi	Service layer + test business
Sprint 6	1 settimana	Implementazione notifiche (SistemaNotifiche + NotificaDAO), refactoring DAO	Sistema notifiche + refactoring
Sprint 7	1 settimana	Creazione State Machine Diagram, riorganizzazione package, implementazione attivaPrenotazione/completaPrenotazione	State Machine + refactoring architettura

Sprint 8	1 settimana	Inizio GUI JavaFX 21: Dashboard, primi controller (VeicoliController, VeicoloFormController)	Prima versione GUI funzionante
Sprint 9	1 settimana	Implementazione GUI prenotazioni: nuova prenotazione, gestione prenotazioni, integrazione con service e notifications	UI prenotazioni + integrazioni
Sprint 10	1 settimana	Implementazione GUI manutenzioni: creazione manutenzione, NuovaManutenzioneController	UI manutenzioni completa
Sprint 11	1 settimana	Creazione Sequence Diagram, refinement flussi prenotazione/manutenzione, bug fixing generale	Sequence Diagram + codice stabilizzato
Sprint 12	1 settimana	Refactoring finale, revisione test, preparazione documenti finali e demo	Codice consolidato + documentazione + demo

*sprint 11 e 12 sono momentaneamente ipotizzati

2.7 Controllo del processo

- Revisione tecnica a fine sprint tra i due membri del team.
- Retrospettiva breve per identificare problemi e miglioramenti.
- Aggiornamento del Project Plan in caso di deviazioni significative.

3. Organizzazione del progetto

3.1 Relazioni organizzative e contesto

Il progetto FleetManager è sviluppato nell'ambito del corso di Ingegneria del Software A.A. 2025/26, sotto la supervisione del Prof. Angelo Gargantini e della Dott.ssa Silvia Bonfanti, che rappresentano l'organizzazione madre e l'ente valutatore.

Il team di progetto è composto da Pietro Plati e Francesco Basis, che operano come organizzazione utente e di sviluppo, gestendo in autonomia analisi, progettazione, implementazione, testing e documentazione.

Il progetto ha relazioni con:

- Docenti del corso, che forniscono linee guida, feedback e validazione del project plan;
- Repository GitHub, utilizzato come piattaforma di collaborazione, controllo versione e verifica dei progressi;
- Strumenti software di supporto (Eclipse, Papyrus, Maven, SonarQube, H2 Database, PMD, Stan4J, JavaFX 21), che rappresentano le risorse operative per lo sviluppo e la validazione.

3.2 Coinvolgimento degli utenti

I potenziali utenti del sistema FleetManager sono:

- il Manager, responsabile della gestione della flotta aziendale (prenotazioni, manutenzioni, scadenze);
- i Driver, che prenotano e utilizzano i veicoli.

Durante lo sviluppo, questi ruoli vengono simulati dai membri del team stesso, che ne analizzano i requisiti funzionali e ne verificano il corretto comportamento attraverso casi d'uso e test.

Le informazioni e i servizi forniti dagli "utenti simulati" includono:

- l'elenco dei requisiti funzionali;
- i flussi di utilizzo e le regole di business;
- i dati di test e di simulazione per la validazione del software.

3.3 Struttura del team

Il team è autonomo e cooperativo, composto da due membri con ruoli definiti ma complementari.

La comunicazione avviene in modo continuo tramite:

- GitHub Issues e Pull Request (per assegnare e tracciare attività);
- Canale Discord privato per discussione tecnica e sincronizzazione giornaliera;
- Meeting settimanali per verifica milestone e retrospettiva.

Entrambi i membri contribuiscono in modo attivo a:

- definizione e affinamento dei requisiti;
- implementazione delle funzionalità principali;
- revisione incrociata del codice e dei documenti.

3.4 Ruoli e responsabilità

Il team è composto da due membri che hanno operato con un modello di collaborazione paritaria e senza distinzione di ruolo formale.

Tutte le attività, modellazione UML, definizione dei requisiti, sviluppo Java, configurazione Maven e H2, testing JUnit, refactoring, redazione documentazione e realizzazione della GUI sono state svolte insieme, con continua revisione incrociata.

L'assenza di ruoli differenziati ha permesso di mantenere una comunicazione continua, una revisione reciproca costante e una distribuzione equilibrata del carico di lavoro, garantendo qualità e coerenza in tutte le fasi del progetto.

3.5 Formazione e competenze

Il team ha svolto una formazione preliminare su:

- uso di Papyrus UML per la modellazione;
- gestione di progetti Maven e configurazione delle dipendenze;
- test automatizzati con JUnit 5;
- utilizzo di Git e GitHub per versionamento e code review.

3.6 Struttura organizzativa e flusso decisionale

Le decisioni principali (approvazione requisiti, milestone, merge su main) vengono prese congiuntamente da entrambi i membri.

Il processo decisionale segue tre livelli:

1. Pianificazione (definizione obiettivi sprint e assegnazione attività)
2. Esecuzione (sviluppo e testing autonomo)
3. Revisione (code review e validazione milestone da parte di entrambi)

3.7 Sintesi

L'organizzazione del progetto FleetManager è quindi strutturata su un modello collaborativo a due membri, con ruoli bilanciati e responsabilità condivise.

L'approccio è coerente con il principio espresso da Van Vliet, secondo cui l'organizzazione del team e la definizione dei ruoli sono fondamentali per garantire efficienza, comunicazione e qualità nel ciclo di sviluppo.

4. Standard, linee guida e procedure

4.1 Disciplina di lavoro

Il progetto FleetManager adotta una forte disciplina di lavoro, basata su procedure condivise e verificabili, in modo che ciascun membro del team segua gli stessi standard, convenzioni e modalità operative. Tutti gli standard sono documentati nel repository GitHub e applicati quotidianamente tramite strumenti automatici (Maven, Git, SonarQube, Papyrus, JUnit).

L'obiettivo è assicurare uniformità, tracciabilità e qualità costante tra codice, documentazione e diagrammi UML, come richiesto dalle linee guida del corso di Ingegneria del Software e dai principi del Van Vliet.

4.2 Standard di codifica

- Linguaggio: Java 17, con le convenzioni ufficiali Oracle per la formattazione e la nomenclatura (camelCase per variabili e metodi, PascalCase per classi, UPPER_CASE per costanti).
- Struttura dei package:
 - it.fleetmanager.model
Contiene tutte le classi di dominio, come Utente, Veicolo, Prenotazione, Scadenza, Manutenzione, Notifica e le altre entità del sistema.
 - it.fleetmanager.repository
Raccoglie la logica di persistenza e accesso ai dati.
È suddiviso in:
 - dao: interfacce DAO per ciascuna entità
 - impl: implementazioni delle DAO per il database H2
 - util: classi di supporto (DatabaseManager, DatabaseTestUtils, DatabaseSeeder, ecc.)
 - it.fleetmanager.service
Contiene la logica applicativa del sistema.
È suddiviso in:
 - interfaces: interfacce dei vari servizi
 - impl: implementazioni dei gestori (GestoreLogin, GestorePrenotazioni, GestoreScadenze, GestoreManutenzioni, SistemaNotifiche, ecc.)
 - it.fleetmanager.ui
Contiene l'interfaccia grafica JavaFX 21.
Include sottopackage dedicati alle diverse sezioni dell'applicazione.
 - it.fleetmanager.util
Raccoglie classi di utilità generali utilizzate in diversi punti del progetto.
- Commenti e documentazione interna:

- Ogni classe e metodo pubblico deve contenere JavaDoc.
 - Tutte le variabili significative devono avere un commento esplicativo.
- Controllo di qualità del codice:
 - Analisi statica con SonarQube, PMD e Stan4J.
 - Complessità ciclomatica monitorata periodicamente.

4.3 Standard per la documentazione

Struttura

Tutta la documentazione è salvata nella cartella /documenti/ del repository, organizzata nei seguenti file PDF/Markdown:

1. project_plan.pdf – piano di progetto (questo documento)
2. gestione_progetto.pdf – ciclo di vita, configuration e people management
3. requisiti.pdf – requisiti funzionali e non funzionali (IEEE 830)
4. design.pdf – diagrammi UML, architettura, design pattern
5. testing.pdf – test di unità e copertura
6. maintenance.pdf – attività di refactoring e manutenzione

Regole

- Ogni modifica sostanziale deve essere accompagnata da un commit dedicato e da un commento descrittivo.
- Le versioni principali vengono contrassegnate da tag semantici (v1.0, v1.1, ecc.).
- La qualità della documentazione viene valutata secondo tre criteri:
 1. Completezza – tutti i punti richiesti dal piano Van Vliet sono coperti.
 2. Coerenza – i diagrammi e il codice riflettono i requisiti aggiornati.
 3. Aggiornamento – il documento viene aggiornato alla chiusura di ogni sprint.

4.4 Standard di modellazione UML

- Tutti i diagrammi UML sono creati e mantenuti con Papyrus (formato .uml e .di).
- Tipi di diagrammi obbligatori:
 - Use Case Diagram – descrizione dei requisiti funzionali
 - Class Diagram – struttura delle entità principali
 - State Machine Diagram – stati del veicolo
 - Sequence Diagram – effettuare una prenotazione
 - Activity Diagram – flusso prenotazione e restituzione
 - Component e Package Diagram – organizzazione architetturale
- Ogni modifica del codice che impatta la logica del dominio richiede sincronizzazione del diagramma UML nel commit.

4.5 Procedure operative

- Controllo di versione: ogni attività è tracciata su GitHub tramite branch dedicato e pull request con review obbligatoria.
- Gestione delle dipendenze: centralizzata tramite Maven (pom.xml), con almeno una dipendenza esterna.
- Build e test: eseguiti tramite mvn clean test.
- Gestione configurazione: GitHub funge da sistema CM; le issue e i PR documentano le modifiche.

4.6 Strumenti di supporto

Strumento	Funzione
Eclipse IDE	Sviluppo e debug del codice java
Papyrus UML	Modellazione e generazione parziale del codice
Maven	Gestione build e dipendenze
JUnit 5	Testing automatico
Log4j2	Logging di sistema
H2 Database	Database embedded
GitHub	Versionamento, issue tracking e collaborazione
SonarQube / PMD / Stan4J	Analisi statica e qualità del codice
JavaFX 21	Parte di GUI

4.7 Aggiornamento e controllo

L'aggiornamento degli standard e della documentazione è continuo:

- ogni fine sprint prevede la revisione del codice, dei diagrammi e dei documenti;
- eventuali modifiche agli standard devono essere approvate da entrambi i membri e registrate nel file changelog.md;
- il rispetto degli standard è verificato tramite:
 - linting automatico nella build Maven;
 - code review reciproca;
 - confronto tra documentazione e codice in fase di milestone.

4.8 Sintesi

Gli standard e le procedure di FleetManager garantiscono coerenza, qualità e aggiornamento costante del progetto. Come indicato da Van Vliet, “una forte disciplina di lavoro e accordi chiari sulla documentazione” sono essenziali per il successo di un

progetto software: FleetManager traduce questo principio in pratiche operative concrete, integrate nel flusso di lavoro quotidiano.

5. Attività di gestione

5.1 Obiettivi di gestione

Le attività di gestione del progetto FleetManager hanno come scopo principale il monitoraggio costante dello stato di avanzamento e la verifica del rispetto dei vincoli di tempo, qualità e ambito.

Seguendo quanto indicato dal Van Vliet, il team deve:

- produrre relazioni periodiche sull'andamento del lavoro e sull'evoluzione delle milestone;
- bilanciare requisiti, tempi e risorse per garantire la consegna nei termini previsti;
- rilevare tempestivamente eventuali deviazioni e definire azioni correttive.

5.2 Struttura di controllo

Poiché il team è composto da due persone, la gestione del progetto segue un modello collaborativo e paritario, in cui entrambi i membri partecipano attivamente a tutte le attività di analisi, sviluppo, modellazione, test e documentazione. Le decisioni operative e progettuali vengono prese congiuntamente, garantendo una revisione continua e reciproca del lavoro svolto.

Per mantenere un efficace controllo dell'avanzamento, il team effettua frequenti momenti di confronto, spesso su base giornaliera, durante i quali vengono discussi:

- lo stato dei task in corso,
- eventuali problemi tecnici o organizzativi,
- l'allineamento rispetto alle milestone pianificate,
- le attività da svolgere nel breve termine,
- i risultati dei test e del refactoring,
- eventuali aggiornamenti ai diagrammi UML o alla documentazione.

In aggiunta, una volta a settimana viene condotta una revisione strutturata dello sprint, con l'obiettivo di verificare la coerenza complessiva del lavoro, aggiornare la pianificazione e identificare eventuali criticità.

Tutte le attività e i progressi sono registrati e monitorati tramite il GitHub Project Board, organizzato nelle colonne *To Do*, *In Progress*, *In Review* e *Done*, che funge da strumento di controllo visivo e supporto alla gestione Agile del progetto.

5.3 Metriche di avanzamento

L'avanzamento del progetto viene monitorato tramite:

- Numero di issue chiuse rispetto al totale previsto per lo sprint;
- Numero di commit e PR integrati nella branch principale;
- Percentuale di test JUnit superati;
- Copertura del codice ≥ 70 %;

Ogni sprint si conclude con una breve retrospettiva, in cui si valuta se gli obiettivi sono stati raggiunti e si aggiornano le priorità del backlog.

5.4 Relazioni periodiche

Nel contesto di un team di due persone che lavora in modo continuo e collaborativo, la gestione dell'avanzamento non avviene tramite report formali separati, ma attraverso un sistema di monitoraggio incrementale e costante.

L'avanzamento dello sprint viene verificato quotidianamente mediante:

- discussioni dirette e confronti frequenti sul lavoro svolto,
- aggiornamenti del GitHub Project Board (To Do → In Progress → In Review → Done),
- commit che documentano lo stato delle attività,
- revisione reciproca del codice, dei diagrammi UML e dei test.

Alla fine di ogni sprint viene comunque effettuata una sintesi informale, registrata tramite aggiornamenti del Project Board e del Changelog, nella quale si verifica:

1. le attività completate e quelle da completare nello sprint successivo;
2. lo stato dei test e le eventuali correzioni effettuate;
3. il progresso rispetto alle milestone;
4. eventuali problemi emersi e loro risoluzione;
5. le decisioni operative prese per orientare il lavoro successivo.

Questo approccio, tipico delle metodologie Agile, ha permesso di mantenere una visione costante e condivisa dell'avanzamento, senza la necessità di documenti formali aggiuntivi.

5.5 Bilanciamento requisiti – tempi – costi

Il progetto viene gestito secondo il principio del triangolo di progetto:

- Requisiti: priorità alle funzionalità fondamentali (gestione veicoli, prenotazioni, scadenze).
- Tempo: rispetto rigoroso delle scadenze di sprint e milestone.
- Costi: misurati in termini di ore /uomo, stimati in circa 35–40 ore per persona.

Quando emerge un conflitto tra tempo e ambito, viene adottata la seguente politica:

- non ridurre la qualità, ma ridimensionare temporaneamente la quantità di funzionalità non critiche;
- aggiornare la pianificazione, notificando la variazione nel report dello sprint.

5.6 Indicatori di successo

Un'attività di gestione è considerata efficace se:

- tutte le milestone vengono raggiunte entro la data prevista;
- non si verificano regressioni funzionali nei test;
- la documentazione è coerente con il codice;
- i membri del team mantengono un flusso di commit costante e coordinato.

5.7 Sintesi

Le attività di gestione del progetto FleetManager assicurano controllo continuo, trasparenza e bilanciamento tra obiettivi e risorse. In conformità al Van Vliet, il team garantisce relazioni periodiche sullo stato di avanzamento e mantiene un equilibrio dinamico tra requisiti, tempi e costi, condizione indispensabile per la riuscita del progetto.

6. Rischi

6.1 Identificazione precoce dei rischi

Ogni progetto di sviluppo software comporta inevitabilmente delle incertezze: errori tecnici, ritardi, mancanza di risorse o problemi organizzativi.

Seguendo il principio espresso da Van Vliet “i rischi devono essere diagnosticati precocemente e devono essere previste misure per affrontarli” il team ha condotto un’analisi preventiva per identificare i possibili rischi del progetto FleetManager e pianificare azioni di mitigazione. L’obiettivo è ridurre l’impatto sul rispetto delle scadenze, sulla qualità e sulla funzionalità complessiva del sistema.

6.2 Classificazione dei rischi

I rischi vengono suddivisi in quattro categorie principali:

- Tecnici: problemi legati a strumenti, librerie, configurazioni o errori di integrazione tra moduli;
- Organizzativi: difficoltà di coordinamento, comunicazione o gestione dei tempi di lavoro
- Operativi: problemi legati ai test, alla qualità del codice o all’aggiornamento della documentazione;
- Di pianificazione: ritardi nella consegna delle milestone o stima errata del carico di lavoro.

6.3 Tabella di analisi dei rischi

Rischio	Categoria	Probabilità	Impatto	Azioni di mitigazione
Ritardo nell'integrazione dei moduli (es. prenotazioni, scadenze)	Tecnico	Media	Alta	Integrazione continua con test incrementali e commit frequenti.
Difficoltà nell'uso di Papyrus per la sincronizzazione UML con il codice	Tecnico	Media	Media	Formazione iniziale e creazione di template standard per i diagrammi.

Sovraccarico di lavoro o indisponibilità temporanea di un membro del team	Organizzativo	Media	Alta	Pianificazione bilanciata delle attività e backup reciproco tra Pietro e Francesco.
Errori o incompletezza nella documentazione	Operativo	Media	Media	Aggiornamento dei documenti a ogni sprint e revisione incrociata.
Test JUnit non sufficienti o mancata copertura minima	Operativo	Bassa	Alta	Definizione di casi di test in parallelo all'implementazione.
Problemi di compatibilità o configurazione Maven	Tecnico	Media	Media	Verifica costante del file pom.xml e uso di versioni stabili delle librerie.
Rallentamenti dovuti a strumenti o hardware personale non aggiornato	Tecnico	Bassa	Media	Backup periodico e test incrociati su più macchine.
Mancato rispetto delle milestone intermedie	Pianificazione	Media	Alta	Timeboxing rigido, verifica settimanale e aggiornamento del project board.
Incoerenza tra codice e diagrammi UML	Operativo	Media	Media	Revisione congiunta diagrammi–codice al termine di ogni sprint.

7. Personale

7.1 Struttura del team

Il progetto FleetManager è sviluppato da un team composto da due membri che operano in modo **collaborativo, paritario e continuo** durante tutte le fasi del ciclo di vita del software.

Le attività di analisi, modellazione UML, implementazione, test, documentazione e revisione sono state svolte insieme, con decisioni condivise e una revisione reciproca costante.

7.2 Competenze richieste

Il progetto richiede competenze diversificate, sia tecniche sia organizzative. Ogni membro ha consolidato o acquisirà le seguenti abilità:

Area	Competenze richieste	Modalità di acquisizione
Analisi e Requisiti	Identificazione requisiti funzionali e non funzionali, tracciamento use case	Studio delle slide del corso, confronto interno
Progettazione UML	Use Case, Class, State, Sequence, Activity, Component, Package Diagram (Papyrus)	Esercitazioni e lavoro incrementale sul modello
Sviluppo Software	Java 17, Eclipse, Maven, DAO Pattern, Log4j2, JavaFX 21	Implementazione diretta e revisione reciproca
Testing & QA	JUnit 5, metriche McCabe, analisi statica (PMD / SonarQube / Stan4J)	Autovalutazione e test iterativi
Configuration Management	Uso avanzato di GitHub (branch, PR, issue, tag)	Applicazione pratica nel ciclo di sviluppo
Documentazione	Redazione Project Plan e altri 5 documenti richiesti	Aggiornamento continuo a fine sprint

7.3 Distribuzione temporale del personale

Anche se il team rimane costante, la concentrazione dell'impegno varia secondo le fasi del progetto:

Fase	Durata stimata
Analisi e Requisiti (Sprint 0–1)	~15 ore
Progettazione UML (Sprint 1–2)	~20 ore

Implementazione (Sprint 2–4)	~40 ore
Testing e QA (Sprint 4–5)	~15 ore
Documentazione e Presentazione (Sprint 5)	~10 ore
Totale stimato	100 ore ($\approx$ 50 ore/persona)

7.4 Disponibilità e coordinamento

Il team ha lavorato con un modello di collaborazione continuo e flessibile, caratterizzato da frequenti momenti di confronto sia in presenza che tramite strumenti online.

La comunicazione e il coordinamento sono stati regolari e costanti durante l'intero progetto.

- Frequenza di lavoro: sessioni di sviluppo e revisione svolte più volte alla settimana, integrate da momenti di confronto quotidiano sui progressi.
- Meeting interni: incontri periodici dedicati alla pianificazione dello sprint, alla verifica delle attività e al monitoraggio delle milestone.
- Strumenti di comunicazione: GitHub (issue, commit, Pull Request) come punto di riferimento per la collaborazione tecnica, e Discord per la sincronizzazione rapida e le decisioni operative.
- Decisioni tecniche: sempre prese congiuntamente, tramite confronto diretto e valutazione condivisa delle alternative.

Questo approccio collaborativo ha garantito una gestione efficace del progetto, un rapido scambio di informazioni e una costante allineamento tra i membri del team.

7.5 Formazione e aggiornamento

Le competenze richieste sono in larga parte già acquisite grazie alle esercitazioni di laboratorio. Tuttavia, per garantire uniformità metodologica, il team prevede:

- Autoformazione su Papyrus UML, Maven build management e JavaFX 21;
- Verifica incrociata delle procedure di testing e qualità;
- Revisione documentale reciproca prima di ogni milestone.

7.6 Sintesi

Il personale coinvolto nel progetto FleetManager è composto da due sviluppatori che coprono l'intero ciclo di vita del software, con ruoli e competenze equilibrate.

L'organizzazione è stabile, le responsabilità sono condivise e l'impegno è distribuito uniformemente. Questo assetto garantisce la continuità del lavoro e la possibilità di mantenere un elevato livello di qualità e coordinamento in tutte le fasi del progetto.

8. Metodi e tecniche

8.1 Metodi applicati lungo il ciclo di vita

Il progetto FleetManager segue un processo iterativo-incrementale con fasi successive ma parzialmente sovrapposte.

Durante ogni iterazione vengono applicati metodi specifici per ciascuna fase dell'ingegneria del software:

Fase	Metodi e strumenti principali
Analisi dei requisiti	Metodo IEEE 830 per la definizione dei requisiti funzionali e non funzionali; raccolta tramite brainstorming e formalizzazione in Use Case Diagram (Papyrus).
Progettazione	Modellazione UML completa (Use Case, Class, Sequence, Activity, State, Component e Package Diagram); architettura multilayer composta da Model, Repository (DAO), Service e interfaccia grafica JavaFX 21.
Implementazione	Programmazione in Java 17 con Eclipse e gestione dipendenze via Maven; codice organizzato in moduli (model, service, repository, ui, test); logging tramite Log4j2.
Testing	JUnit 5 per test di unità e integrazione; misurazione copertura con Maven Surefire; metriche di qualità (SonarQube, PMD, Stan4J).
Manutenzione	Refactoring progressivo e aggiornamento continuo dei diagrammi UML; gestione modifiche con branch dedicati e pull request revisionate.

8.2 Tecniche di gestione della configurazione

Il controllo di versione è gestito interamente con GitHub. Le principali convenzioni operative sono:

- Branching model:
 - Main: stabile e rilasciabile;
 - Develop: sviluppo attivo;
 - GUI: con l'integrazione della parte grafica;
- Pull Request con revisione reciproca obbligatoria.
- Tag semantici per le milestone (v1.0.0, v1.1.0, ...).
- Issues per la tracciabilità dei cambiamenti e dei bug.
- File .gitignore per escludere artefatti generati (/target, *.class, *.log).

La configurazione del progetto (struttura cartelle, dipendenze, plugin) è centralizzata nel file pom.xml. Ogni modifica sostanziale al build o alle librerie è documentata in changelog.md.

8.3 Ambiente di sviluppo e di prova

Componente	Strumento / versione	Ruolo
IDE	Eclipse 2025-09	sviluppo e debug
Linguaggio	Java 17	implementazione logica
Build / Gestione dipendenze	Maven 3.9+	compilazione e test
Modellazione	Papyrus 2025	diagrammi UML
Grafica	JavaFX 21	tutta la parte di GUI
Database	H2 1.4.x (embedded)	persistenza locale
Testing	JUnit 5	test automatici
QA	SonarQube, PMD, Stan4J	analisi statica e metriche
Version control	GitHub	collaborazione e tracking

Tutti i test vengono eseguiti su macchine personali dei due sviluppatori (Windows 10 e MacOS Tahoe), con JDK 17 e ambiente Maven configurato. L'esecuzione dei test di integrazione non richiede connessione di rete né hardware aggiuntivo.

8.4 Ordine d'integrazione e test

L'integrazione dei componenti segue un approccio bottom-up, per ridurre l'impatto di errori di dipendenza:

1. Componenti di base: classi di dominio (Veicolo, Utente, Prenotazione).
2. Repository / DAO: funzioni CRUD e gestione persistenza.
3. Service Layer: logica applicativa (prenotazioni, scadenze, manutenzioni).
4. Gestione notifiche.
5. GUI JavaFX 21.
6. Testing JUnit completo e verifica funzionale.

8.5 Procedure di test e accettazione

I test sono stati realizzati principalmente tramite JUnit 5 e sfruttano un database H2 in memoria, configurato tramite le classi di supporto DatabaseTestUtils.

Sono state applicate le seguenti tipologie di test:

- Test unitari: Riguardano i metodi delle componenti fondamentali del sistema, in particolare:
 - DAO (inserimento, aggiornamento, eliminazione, ricerca)
 - Service (GestorePrenotazioni, GestoreLogin, GestoreScadenze, GestoreManutenzioni)
 - Validazione stati e transizioni dei veicoli
- Test di integrazione: Verificano il corretto funzionamento congiunto di più moduli:
 - interazione Service - Repository - H2 in-memory
 - comportamento del sistema su scenari completi (es. prenotazione + cambio stato + notifica)
- Test di sistema: Simulano casi d'uso reali come:
 - creazione e gestione di una prenotazione
 - attivazione e completamento prenotazione
 - generazione automatica delle scadenze e delle notifiche
 - creazione e completamento di una manutenzione
- Test di accettazione: Corrispondono alla validazione del comportamento del sistema rispetto ai requisiti funzionali principali:
 - prenotazione corretta di un veicolo disponibile
 - rifiuto prenotazione su veicolo occupato
 - aggiornamento stato veicolo secondo la State Machine
 - generazione notifiche su eventi rilevanti (prenotazioni, scadenze, manutenzioni)
 - corretto popolamento e lettura dal database

Struttura dei test: ogni test include:

1. Identificativo e descrizione sintetica
2. Input, precondizioni e comportamento atteso
3. Risultato effettivo (PASS / FAIL)
4. Riferimento ai requisiti funzionali e non funzionali

8.6 Documentazione tecnica prodotta

Durante le fasi di sviluppo vengono generati automaticamente i seguenti artefatti:

- JavaDoc con `mvn javadoc:javadoc`;
- report JUnit (XML/HTML);
- report PMD, SonarQube e Stan4J;
- diagrammi UML aggiornati in Papyrus.

Tali documenti vengono archiviati nel repository sotto le cartelle `/target/reports/` e `/documenti/`, garantendo tracciabilità e verificabilità costante.

8.7 Sintesi

I metodi e le tecniche adottati assicurano un processo rigoroso, documentato e controllato in ogni fase. Il sistema di versionamento GitHub e gli strumenti Maven-JUnit permettono una gestione efficiente della configurazione e del testing.

9. Garanzia di qualità

9.1 Obiettivo

L'obiettivo del piano di garanzia della qualità è assicurare che il software FleetManager soddisfi i requisiti di correttezza, affidabilità, manutenibilità, efficienza e chiarezza previsti nelle specifiche.

In linea con Van Vliet, la qualità viene perseguita attraverso una combinazione di:

- procedure organizzative (revisioni, test, controllo di configurazione),
- strumenti di verifica automatica,
- metriche quantitative di valutazione del codice e dei processi.

9.2 Organizzazione della qualità

Poiché il team è composto da due membri, la garanzia della qualità è integrata nel processo stesso.

Le decisioni sulla qualità vengono prese in comune e discusse al termine di ogni sprint, sulla base dei risultati dei test e delle analisi statiche.

9.3 Procedure di garanzia

1. Code Review: ogni pull request deve essere revisionata dall'altro membro prima del merge su main.
2. Testing automatico: tutti i moduli devono essere coperti da test JUnit 5. La build Maven fallisce se i test non passano.
3. Analisi statica: uso di SonarQube, PMD e Stan4J per rilevare code smells, duplicazioni e violazioni di standard.
4. Controllo UML: coerenza periodica tra diagrammi Papyrus e codice Java.
5. Retrospettiva di qualità: verifica finale a ogni sprint sul rispetto dei requisiti di qualità prefissati.

9.4 Requisiti di qualità

Il progetto adotta i seguenti criteri di qualità misurabili:

Attributo	Descrizione	Misura / Strumento
Correttezza	Il sistema produce risultati conformi ai requisiti.	100 % test JUnit superati.
Affidabilità	Assenza di crash, errori di stato o inconsistenze.	Test d'integrazione e stress test.
Manutenibilità	Codice modulare, leggibile e documentato.	Analisi SonarQube / PMD / Stan4J, JavaDoc completo.
Efficienza	Tempo di risposta e uso risorse proporzionati alla scala del problema.	Misure empiriche su casi di carico simulati.
Portabilità	Esecuzione su diversi sistemi Java SE 17.	Test su macchine differenti (Windows / Server 2022).
Coerenza documentale	Allineamento tra requisiti, design e implementazione.	Revisione documenti a fine sprint.

9.5 Metriche di valutazione

Per il progetto FleetManager sono state adottate metriche qualitative e quantitative basate sugli strumenti di analisi utilizzati (SonarQube, PMD, Stan4J). Le metriche hanno guidato sia il refactoring sia la validazione finale del codice.

Le principali metriche applicate sono:

- Complessità ciclomatica (McCabe):
Monitorata tramite SonarLint/SonarQube e mantenuta entro valori accettabili per garantire leggibilità e manutenibilità.
- Code smells e vulnerabilità:
Rilevati da SonarQube, PMD Stan4J; l'obiettivo è stato ridurre progressivamente tali elementi tramite refactoring continuo.
- Regole PMD:
Applicazione delle principali regole di qualità (unused variables, classi eccessivamente complesse, metodi troppo lunghi, duplicazioni, ecc.) e correzione dei warning.
- Analisi architetturale (Stan4J):
Verifica delle dipendenze tra package, rispetto dell'architettura a livelli e assenza di cicli indesiderati.
- Documentazione del codice:
Percentuale di metodi e classi documentate tramite commenti significativi (non misurata numericamente, ma verificata durante le revisioni interne).
- Esecuzione dei test:
Stabilità della suite JUnit e assenza di test falliti al momento della consegna.
- Qualità dei commit e del repository:
Chiarezza dei messaggi di commit, uso di branch e Pull Request, e corrispondenza tra codice e UML aggiornato

9.6 Ambiente di controllo qualità

- GitHub: verifica continua tramite workflow e log di commit.
- Maven Surefire: esecuzione automatica dei test.
- Papyrus: modello UML allineato con il codice.
- SonarQube / PMD / Stan4J: analisi statica e report.
- JUnit 5: verifica automatica di regressione.

Ogni strumento contribuisce a un diverso livello di qualità (processo, codice o documentazione).

9.7 Audit e revisione

Ogni fase del progetto include una revisione interna della qualità, con checklist predefinita:

- completezza requisiti;
- consistenza UML–codice;
- percentuale test superati;
- aggiornamento documentazione.

In caso di deviazioni, viene aperta un'issue GitHub e pianificata la correzione nel successivo sprint.

9.8 Sintesi

Il sistema di garanzia della qualità di FleetManager è integrato nel flusso di sviluppo Agile e basato su controlli automatici, metriche oggettive e revisioni manuali. In questo modo, il software viene costantemente verificato per soddisfare i requisiti funzionali e di qualità dichiarati.

10. Risorse

10.1 Obiettivo

Questa sezione elenca tutte le risorse necessarie allo sviluppo, alla verifica e alla consegna del progetto FleetManager. Comprende:

- risorse hardware e infrastrutturali,
- strumenti software di supporto,
- e il personale impiegato durante le diverse fasi del processo di sviluppo.

10.2 Risorse hardware

Le risorse hardware richieste per il progetto sono modeste, poiché lo sviluppo e il test avvengono in locale su macchine personali dei membri del team.

Risorsa	Specifiche	Utilizzo
Laptop / PC	CPU Intel i5, 16 GB RAM, SSD 1TB	Sviluppo, test, documentazione
Storage esterno o cloud (GitHub)	Spazio repository remoto	Versionamento, backup, collaborazione

10.3 Risorse software

Categoria	Strumento / Versione	Funzione
IDE di sviluppo	Eclipse 2025-09 (Java 17)	Programmazione, debug
Gestione dipendenze / build	Maven 3.9+	Compilazione, test e packaging
Linguaggio	Java 17	Implementazione
Database embedded	H2 1.4.x	Persistenza dati locale
Testing	JUnit 5, Maven Surefire	Test unitari e report automatici
Interfaccia grafica	JavaFX 21	GUI, controller e gestione viste FXML
Logging	Log4j2	Monitoraggio e tracciamento eventi
Analisi statica	SonarQube, PMD, Stan4J	Controllo qualità e metriche
Modellazione UML	Papyrus 2025	Diagrammi e design architetturale
Versionamento e collaborazione	Git + GitHub	Controllo versione e revisione
Documentazione	Markdown + ReportLab (PDF)	Stesura e formattazione documenti
Comunicazione interna	Discord	Coordinamento del team

10.4 Risorse umane (personale)

Fase del processo	Risorse coinvolte	Attività principali
Analisi e requisiti	Pietro e Francesco (collaborazione congiunta)	Raccolta requisiti, discussione casi d'uso, formalizzazione secondo IEEE 830.
Progettazione UML	Pietro e Francesco (co-modellazione)	Creazione e aggiornamento dei diagrammi UML (Use Case, Class, State, Sequence, Activity, Package) con Papyrus.
Implementazione	Pietro e Francesco (co-sviluppo)	Sviluppo dei moduli core (model, repository, service, UI), configurazioni Maven, gestione database H2, integrazione JavaFX 21.
Testing e QA	Pietro e Francesco (testing incrociato)	Test JUnit su DAO e Service, uso di Maven Surefire, analisi statica con SonarQube/PMD/Stn4J, revisione codice e metriche.
Documentazione finale	Pietro e Francesco (redazione condivisa)	Redazione del Project Plan, aggiornamento requisiti e design, preparazione materiale per la presentazione.

10.5 Cicli di CPU e risorse di calcolo

Il progetto non richiede risorse computazionali elevate. Le prove di stress vengono eseguite su laptop di fascia media. Le build Maven e i test JUnit richiedono in media:

- CPU usage: 25–30 % durante la compilazione e i test;
- RAM usage: 400–600 MB per JVM e H2 database;
- Tempo medio di build: < 10 secondi.

10.6 Gestione delle risorse

- GitHub funge da repository centralizzato e da Configuration Management System;
- Papyrus e Maven assicurano coerenza tra design e codice;
- Backup automatici garantiscono sicurezza dei dati e versioni precedenti dei file;
- L'utilizzo delle risorse viene monitorato e bilanciato durante ogni sprint, in base al carico di lavoro.

10.7 Sintesi

Le risorse necessarie per FleetManager sono interamente disponibili e sostenibili per un team di due persone. L'hardware standard, combinato con strumenti software professionali (Eclipse, Maven, Papyrus, GitHub, JUnit), consente di realizzare un processo di sviluppo completo, ripetibile e documentato.

Il dimensionamento è coerente con la scala del progetto e con le linee guida di Van Vliet, che richiedono la definizione di risorse fisiche, logiche e umane per ogni fase del processo.

11. Budget e programma

11.1 Obiettivo

Scopo di questa sezione è definire:

- la distribuzione delle risorse e del tempo tra le attività principali del progetto,
- la stima del budget teorico (in ore e valore economico simulato),
- e la programmazione temporale delle fasi di lavoro, come richiesto dal capitolo 7 del Van Vliet.

11.2 Stima del tempo complessivo

Il progetto FleetManager è pensato per una durata di circa 10 settimane, in linea con il calendario del corso e la scala del progetto didattico (circa 110/120 ore totali).

Fase del processo	Durata stimata	Periodo indicativo	Output prodotto
Sprint 0 – Setup e struttura iniziale	~ 1 settimana (8–10 ore)	metà ottobre 2025	Creazione repository GitHub, progetto Maven, setup Papyrus, prime cartelle UML, README e Project Plan iniziale
Sprint 1 – Requisiti e modellazione base	~ 1 settimana (12–15 ore)	fine ottobre 2025	Analisi requisiti, casi d'uso, attori, Use Case Diagram, prime versioni Class Diagram
Sprint 2 – Progettazione UML avanzata	~ 1 settimana (12–15 ore)	inizio novembre 2025	Refining Class Diagram, aggiunta package, relazioni complete, definizione dei modelli per DAO/Service, aggiornamento documenti
Sprint 3 – Implementazione core (DAO + model + DB)	~ 2 settimane (25–30 ore)	metà → fine novembre 2025	Classi model, strutture DAO, H2 database, seeder JSON, fix Class Diagram, test DAO (JUnit)
Sprint 4 – Servizi e logica di business	~ 1 settimana (15–20 ore)	fine novembre → inizio dicembre 2025	GestoreLogin, GestorePrenotazioni, GestoreScadenze, GestoreManutenzioni, notifiche, test Service
Sprint 5 – Introduzione GUI (JavaFX) e integrazione	~ 1 settimana (15–20 ore)	inizio dicembre 2025	SceneManager, Login, Dashboard Manager/Driver, CRUD veicoli, prime viste prenotazioni/manutenzioni
Sprint 6 – Funzionalità avanzate GUI + state machine	~ 1 settimana (15–20 ore)	metà dicembre 2025	Prenotazioni complete, selezione orari, scadenze automatiche, state machine veicolo, integrazione Service ↔ UI
Sprint 7 – Testing, QA e refactoring	~ 1 settimana (12–15 ore)	fine dicembre 2025	Test incrociati, revisione SonarQube/PMD/Stan4J, metriche, build Maven, fix UX JavaFX
Sprint 8 – Documentazione e presentazione finale	~ 1 settimana (8–10 ore)	fine dicembre → inizio gennaio 2026	Documenti finali, revisione UML, Project Plan completo, test report, preparazione slide

11.3 Budget teorico (stima economica accademica)

Anche se si tratta di un progetto didattico, si può simulare un costo basato sul valore medio di un junior developer (≈ € 25/ora).

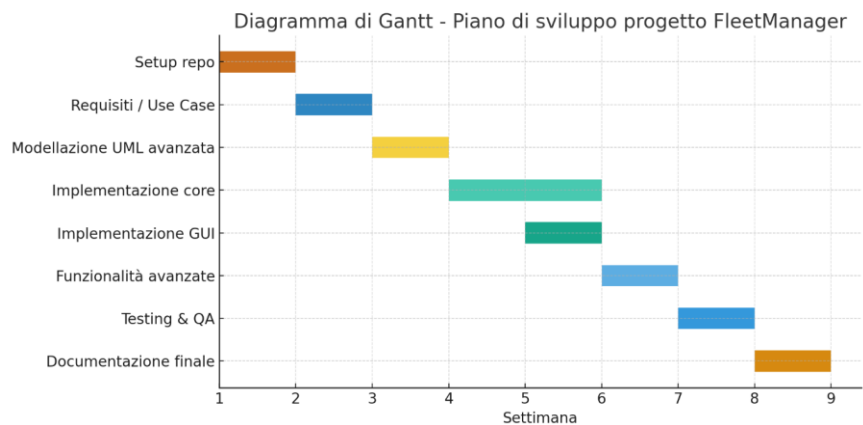
Attività	Ore totali	Costo unitario (€)	Costo stimato (€)
Analisi & Requisiti	12	25	300
Modellazione UML & Design	20	25	500
Implementazione (DAO, Service, DB, GUI)	40	25	1 000
Testing & QA (JUnit, Surefire, SonarLint/PMD)	18	25	450
Documentazione & Presentazione	10	25	250
Totale stimato	100 ore	—	2 500 € (≈ 1 250 €/persona)

Questa stima serve solo a quantificare l'impegno e la ripartizione delle attività, non a rappresentare costi reali.

11.4 Allocazione risorse

Tipo di risorsa	Impiego principale	Fase coinvolta
Hardware	PC + Laptop16 GB RAM + SSD	Tutte
Software	Eclipse, Maven, JavaFX 21, Papyrus 2025, Git/GitHub, JUnit 5, Maven Surefire, Log4j2, SonarLint, PMD, Stan4J	Tutte
Personale	Pietro Plati e Francesco Basis	Tutte
Database H2	Archivio dati embedded per sviluppo, test automatizzati e simulazione della persistenza	Implementazione / Testing

11.5 Pianificazione temporale (diagramma Gantt semplificato)



11.6 Monitoraggio dell'avanzamento

Il monitoraggio dell'avanzamento del progetto è stato condotto in modo continuo e collaborativo. Le milestone e le attività vengono seguite tramite il Project Board di GitHub (colonne *To Do*, *In Progress*, *In Review*, *Done*), aggiornato regolarmente nel corso dello sviluppo.

Il controllo dell'avanzamento si è basato su:

- verifica quotidiana dello stato delle attività durante le sessioni di lavoro condivise;
- aggiornamento del repository tramite commit frequenti e messaggi descrittivi;
- revisione incrociata del codice e dei diagrammi;
- integrazione continua dei test tramite Maven Surefire.

Gli indicatori qualitativi utilizzati per valutare il progresso sono stati:

- coerenza tra codice, UML e requisiti aggiornati;
- assenza di errori nei test JUnit;
- build Maven eseguite correttamente;
- riduzione progressiva di code smells e warning segnalati da SonarLint/PMD;
- avanzamento delle issue e delle attività sul Project Board.

Questo approccio ha garantito un monitoraggio costante e un allineamento continuo tra i membri del team.

11.7 Sintesi

Il budget di tempo e risorse di FleetManager è perfettamente coerente con la scala del progetto accademico e le linee guida del Van Vliet. La pianificazione in sprint brevi, il controllo continuo tramite GitHub e la distribuzione bilanciata delle ore consentono di monitorare costantemente i progressi e mantenere alta la qualità del prodotto finale.

12. Gestione dei cambiamenti

12.1 Obiettivo

Poiché ogni progetto software evolve nel tempo, la gestione dei cambiamenti ha lo scopo di mantenere il controllo su modifiche a requisiti, codice e documentazione, garantendo che vengano:

- analizzate,
- approvate,
- incorporate in modo coerente,

senza compromettere la qualità o la pianificazione. Il principio guida è quello espresso da Van Vliet: “I cambiamenti sono inevitabili, ma devono essere gestiti in modo ordinato attraverso procedure chiare e tracciabili”.

12.2 Tipologie di cambiamento

Nel progetto FleetManager i cambiamenti possono riguardare:

Tipo di cambiamento	Descrizione	Esempio reale
Requisiti	Aggiunta/modifica funzionalità	Aggiunta delle notifiche di scadenza / prenotazione
Progettazione UML	Allineamento diagrammi con implementazione	Aggiornamento Class Diagram (Utente, Veicolo, Prenotazione, Notifica)
Codice sorgente	Bug fix, refactoring, miglioramenti	Correzione NotificaDAO, miglioramento logica prenotazioni e manutenzioni
Database	Revisione schema H2 usato per i test	Aggiunta Enum, fix PK, test H2 in-memory
Interfaccia grafica (JavaFX)	Aggiornamento dashboard, miglioramenti UI	Fix widget targa, sistemazione layout tabelle
Documentazione	Aggiornamento PDF/Markdown	Revisione Project Plan e documenti finali

12.3 Processo di gestione del cambiamento

Nel progetto FleetManager il processo di gestione dei cambiamenti è stato gestito in modo collaborativo e snello, adatto a un team di due persone che lavora quotidianamente a stretto contatto.

Le modifiche non hanno seguito un flusso burocratico formale, ma sono state comunque trattate con attenzione per garantire coerenza tra codice, UML e documentazione.

1. Identificazione del cambiamento

Un cambiamento nasceva tipicamente da:

- discussioni durante le sessioni di lavoro congiunte (quasi quotidiane),
- revisione del codice o dei diagrammi,
- problemi riscontrati nei test o nell'uso dell'applicazione.

Esempi reali:

- “Serve aggiungere idScadenza nella notifica”,
- “Il diagramma classe va aggiornato dopo la nuova logica di prenotazione”.

2. Valutazione informale dell'impatto

La decisione veniva presa subito dai due membri, considerando rapidamente:

- il tempo necessario alla modifica,
- la necessità di aggiornare diagrammi o test,
- eventuali ripercussioni su parti esistenti del progetto.

Non venivano prodotti documenti formali: la valutazione avveniva a voce.

3. Implementazione della modifica

Una volta concordata:

- uno dei due sviluppatori applicava la modifica (codice, test, UML o documentazione),
- l'altro verificava il risultato nelle sessioni successive.

I commit su GitHub erano utilizzati come tracciamento implicito del cambiamento.

4. Revisione e sincronizzazione

Dopo la modifica:

- l'altro membro controllava codice, funzionamento e consistenza,
- se necessario venivano applicati ulteriori aggiustamenti.

Le decisioni e gli aggiornamenti venivano condivisi immediatamente.

5. Allineamento con UML e documentazione

Ogni cambiamento rilevante sul codice comportava:

- aggiornamento del Class Diagram in Papyrus,
- eventuale modifica dei Sequence Diagram,
- revisione dei documenti Markdown/PDF.

Questo garantiva una documentazione sempre coerente, pur mantenendo un processo leggero.

12.4 Criteri di approvazione

Una richiesta di modifica viene approvata solo se:

- è giustificata da un'esigenza funzionale o qualitativa;
- non compromette il completamento di milestone in corso;
- è accompagnata da test che dimostrano l'assenza di regressioni;
- mantiene coerenza tra UML - codice - documentazione.

12.5 Strumenti di supporto

- GitHub Issues: tracciamento richieste di modifica;
- Git Branch / PR: implementazione e review;
- Papyrus UML: aggiornamento grafico delle modifiche architetturali;
- Maven + JUnit: verifica automatica post-integrazione;
- Changelog.md: registro storico delle modifiche approvate.

12.6 Rischi associati ai cambiamenti

- Inserire modifiche non registrate può portare a code smell, documentazione incoerente e regressioni.
- Per evitare questi rischi, nessuna modifica diretta è ammessa sul branch main; tutto passa da PR revisionate e issue tracciate.

12.7 Incorporazione nel progetto

Dopo l'approvazione e la verifica, la modifica viene:

1. integrata nella branch principale (main),
2. testata globalmente con mvn clean test,
3. segnalata nel changelog,
4. e infine propagata alla documentazione UML e PDF.

12.8 Sintesi

La gestione dei cambiamenti in FleetManager garantisce ordine, tracciabilità e controllo. Ogni modifica è valutata, registrata, verificata e approvata prima dell'integrazione. Come raccomandato da Van Vliet, ciò consente di mantenere il progetto stabile e coerente, riducendo il rischio di superamento di tempi o degrado qualitativo.

13. Consegna

13.1 Obiettivo

La sezione di consegna definisce i deliverable finali del progetto FleetManager, le modalità di presentazione e i criteri di accettazione.

L'obiettivo è assicurare che tutti i prodotti del progetto quali codice, documentazione e modelli, siano completi, coerenti e verificabili, in linea con i requisiti e con gli standard stabiliti nel piano di progetto.

13.2 Deliverable principali

Al termine del progetto, il team consegnerà i seguenti artefatti:

Tipo	Descrizione	Formato / Percorso
Repository GitHub completo	Tutti i file di progetto, codice sorgente, test, documenti e diagrammi UML	https://github.com/Pietroplati/FleetManager
Progetto Eclipse/Maven	Struttura completa e compilabile, con test automatizzati	/codice/
Diagrammi UML Papyrus	Use Case, Class, State, Sequence, Activity, Component e Package Diagram	/uml/
Documentazione PDF	Project Plan, Gestione progetto, Requisiti, Design, Testing, Maintenance	/documenti/
Testing report	Esiti dei test JUnit, coverage, metriche di qualità (SonarLint/PMD)	/documenti/testing.pdf
Presentazione finale	Slide sintetiche (max 15) per l'esame orale	/documenti/presentazione.pptx
Demo eseguibile	Dimostrazione funzionale del sistema (esecuzione locale)	Fornita in sede d'esame

13.3 Modalità di consegna

- La consegna avverrà tramite repository GitHub, condiviso con i docenti garganti e silviabonfanti.
- Tutti i documenti saranno inclusi nella cartella /documenti/ e nominati secondo la convenzione del corso.
- La versione finale sarà identificata con il tag v1.0.0 e sarà immutabile dopo la deadline.
- Eventuali fix successivi (per la tesi o ulteriori sviluppi) saranno pubblicati su branch post-exam.

13.4 Criteri di accettazione

Il progetto sarà considerato completo e accettabile se rispetta i seguenti criteri:

Categoria	Criterio di accettazione	Verifica
Funzionalità	Tutte le funzionalità principali implementate e funzionanti: gestione veicoli, prenotazioni, scadenze, manutenzioni, login, dashboard driver/manager, GUI JavaFX.	Test pratici + demo finale
UML e Design	Diagrammi Use Case, Class, Sequence e State Machine completi e aggiornati, coerenti con l'implementazione effettiva.	Revisione docenti
Qualità del codice	Codice compilabile senza errori, test JUnit principali superati, analisi statica senza criticità bloccanti (SonarQube/PMD/ Stan4J).	Maven + SonarQube/PMD/ Stan4J
Documentazione	Project Plan, Requisiti, Design e Testing completi e aggiornati; coerenza con quanto sviluppato.	Confronto con checklist ufficiale UNI/BG
Versionamento	Repository GitHub chiaro e strutturato: branch develop / gui usati correttamente, commit significativi, merge finale stabile su main.	Verifica manuale del docente su GitHub
Presentazione	Slide concise e complete, dimostrazione chiara del funzionamento del software entro il tempo limite.	Discussione orale finale

13.5 Responsabilità di consegna

Durante la fase di consegna, Pietro Plati e Francesco Basis operano in stretta collaborazione, condividendo tutte le attività di verifica, test e preparazione dei materiali finali. Entrambi partecipano al build conclusivo del progetto, all'esecuzione dei test di validazione e al push definitivo su GitHub, assicurandosi che il repository sia completo, coerente e privo di errori. La documentazione finale e le slide di presentazione vengono redatte e revisionate congiuntamente, in modo da mantenere un allineamento costante tra codice, diagrammi UML e report tecnici.

13.6 Processo di verifica finale

- Completamento sviluppo su branch dedicati
 - Le ultime modifiche vengono completate nei branch develop e gui prima della consegna.
- Merge finale su main
 - Dopo verifica congiunta, il codice stabile viene unificato nel branch principale.
- Esecuzione test e verifica qualità
 - Tutti i test JUnit devono risultare PASS.
 - Analisi statica con SonarLint/PMD senza errori bloccanti.
- Revisione documentazione
 - Aggiornamento finale di Project Plan, Requisiti, UML, Testing e presentazione.
 - Controllo coerenza tra diagrammi e codice.
- Preparazione demo
 - Prova dell'applicazione su laptop (GUI JavaFX, gestione completa del flusso).
 - Verifica funzionamento di tutte le principali funzionalità.
- Consegna finale
 - Upload versione definitiva su GitHub e preparazione alla presentazione orale.

13.7 Valutazione e prospettive future

Il progetto FleetManager costituisce un caso di studio completo per l'applicazione delle metodologie di Ingegneria del Software. Al termine della consegna, il sistema potrà essere:

- esteso con un'interfaccia web o mobile,
- integrato con un database remoto,
- e utilizzato come base per un'eventuale tesi o progetto di ricerca.

13.8 Sintesi

La fase di consegna rappresenta la chiusura formale del progetto. Grazie alla gestione rigorosa del versionamento, alla documentazione strutturata e ai test automatizzati, FleetManager rispetta tutti i criteri di accettazione stabiliti, garantendo un prodotto software completo, verificato e conforme agli standard del corso e del modello Van Vliet.